FullStackCSharp

Guide d'Architecture & Conception Logicielle Fullstack

.NET MAUI + ASP.NET Core Minimal APIs (.NET 8 LTS)

Comprendre les rouages d'une application professionnelle C# à travers des schémas de flux, des diagrammes d'architecture et des fiches ultra-granulaires par action.

Auteur & Concepteur du Projet

Kevin Friart

Ingénierie Logicielle & Architecture Systèmes .NET

11 novembre 2026

Table des matières

1 Jalon 1 : Socle Technique & Initialisation **2**
1.1 Action 1.1 : Création de la Solution & Rôle du Fichier .sln 2
1.2 Action 1.2 : Le Projet de Bibliothèque de Classes (Shared) 5
1.3 Action 1.3 : Création de l'API Backend (ASP.NET Core Minimal API) 7

Chapitre 1

Jalon 1 : Socle Technique & Initialisation

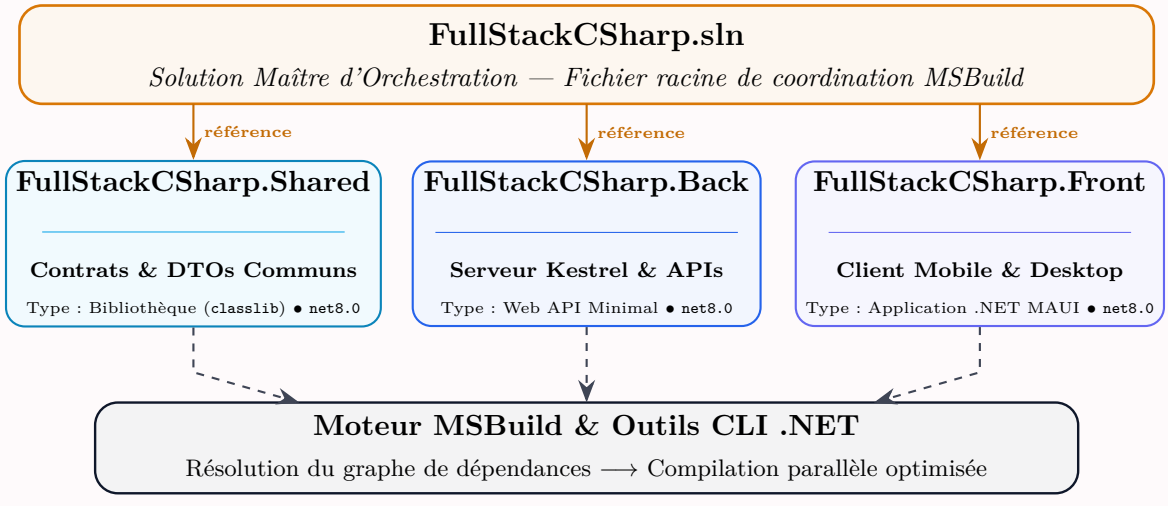
1.1 Action 1.1 : Création de la Solution & Rôle du Fichier .sln

▶ Concept Clé d'Architecture : Qu'est-ce qu'une Solution .NET (.sln) ?

Dans l'écosystème .NET, une **Solution** n'est ni un binaire exécutable, ni une bibliothèque de code. Il s'agit d'un **conteneur d'orchestration** textuel qui référence et coordonne plusieurs projets indépendants (`.csproj`).
Le fichier `.sln` remplit trois fonctions fondamentales pour les outils d'ingénierie (CLI `dotnet`, Visual Studio, Rider, serveurs de CI/CD) :

1. **Graphe de dépendances** : Il référence l'emplacement physique de chaque projet sur le disque et permet à MSBuild de déterminer l'ordre logique de compilation.
2. **Configurations cibles** : Il synchronise les matrices de compilation (`Debug` ou `Release`, architectures `Any CPU`, `x64`, etc.) pour l'ensemble des projets d'un seul bloc.
3. **Point d'entrée unique** : Il permet au développeur d'exécuter `dotnet build` ou `dotnet test` à la racine sans avoir à parcourir chaque sous-répertoire manuellement.

▶ Schéma Conceptuel & Flux : L'Orchestration par la Solution dans l'Architecture



Distinction Conceptuelle : Solution vs Projet vs Package

Niveau	Rôle Principal	Fichier Source	Produit de Sortie
Solution	Orchestration multi-projets, point d'entrée outillage.	`FullStackCSharp.sln`	Aucun binaire (graphe logique).
Projet	Unité de compilation élémentaire contenant le code C#.	`*.csproj`	Assembly (`.dll`) ou Exécutable (`.exe`).
Package	Composant réutilisable redistribué (dépendance).	`*.nupkg` (via NuGet)	Archive binaire versionnée.

▶ Règle d'Ingénierie & Bonne Pratique : Format .sln vs .slnx en .NET 8 / .NET 10

À partir du SDK .NET 9/10, la commande `dotnet new sln` génère par défaut un nouveau format expérimental au format XML (`.slnx`).
Cependant, dans un contexte professionnel de production ciblant **.NET 8 LTS**, il est recommandé de forcer le format standard éprouvé via le paramètre :

```
dotnet new sln -n FullStackCSharp -format sln
```

Cette rigueur garantit une interopérabilité sans faille avec l'ensemble des environnements de développement (Visual Studio 2022, JetBrains Rider, VS Code et les agents d'intégration continue Linux/Windows).

► Questions Fréquentes du Débutant : Questions Fondamentales & Réponses Pédagogiques

1. **Où se trouve la console ? Pourquoi n'y a-t-il pas de projet `FullStackCSharp.Console` dans la solution ?**

   Réponse : En .NET moderne, une API Web ASP.NET Core (`FullStackCSharp.Back`) **est elle-même nativement une application console** (`<OutputType>Exe</OutputType>`). Lorsque vous lancez le serveur avec la commande `dotnet run`, une véritable fenêtre console s'ouvre dans votre terminal. C'est le serveur Kestrel lui-même qui agit directement comme console d'observabilité en affichant en temps réel les requêtes HTTP, le code SQL exécuté et les jetons JWT. Il n'y a donc aucun besoin d'ajouter un 4e projet artificiel.

2. **Pourquoi créer un fichier `.sln` au lieu de juste ouvrir le dossier dans l'éditeur ?**

   Réponse : Ouvrir un dossier permet d'éditer du texte brut, mais le compilateur MSBuild est incapable de deviner automatiquement l'ordre dans lequel compiler vos composants. Le fichier `.sln` fournit à l'outillage la carte officielle du système : il indique formellement que `Shared` doit être compilé avant `Back` et `Front`, et synchronise les options de compilation (`Debug`/`Release`) pour l'ensemble des projets d'un seul bloc.

3. **Pourquoi séparer la solution en 3 projets plutôt que de tout coder dans un seul ?**

   Réponse : C'est la règle d'or de la **séparation des responsabilités** et de la sécurité. Le client mobile ne doit jamais charger les bibliothèques du serveur ni accéder directement à la base de données SQLite centrale. De son côté, le serveur n'a pas à charger le moteur graphique XAML. Seul le projet `Shared` (qui ne contient que des modèles de données DTOs purs et légers) sert de passerelle commune.

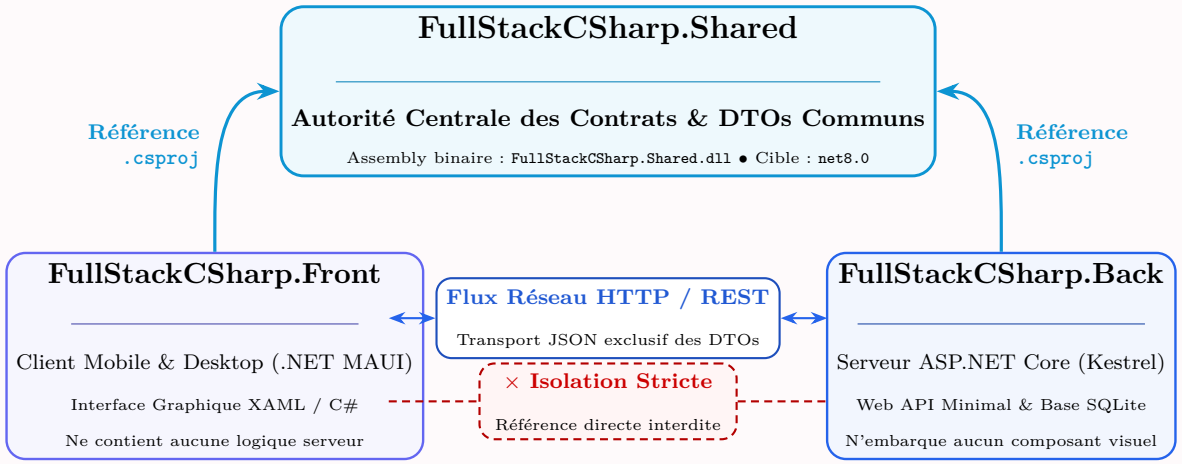
1.2 Action 1.2 : Le Projet de Bibliothèque de Classes (Shared)

▶ Concept Clé d'Architecture : Le Rôle d'une Bibliothèque de Classes (`classlib`) Agnostique

Le premier projet créé dans notre solution est **`FullStackCSharp.Shared`**.
Contrairement aux applications exécutables, une bibliothèque de classes (`classlib`) ne dispose pas de méthode d'entrée `Main()` et ne produit aucun fichier `.exe` : elle est compilée sous la forme d'un assemblage binaire `.dll` (*Dynamic Link Library*) réutilisable.
Dans notre architecture, ce projet occupe une fonction stratégique :

1. **Agnosticisme technologique** : Il est 100% neutre. Il ne dépend ni des bibliothèques graphiques de l'application cliente (.NET MAUI, XAML), ni du serveur web (ASP.NET Core, Kestrel), ni de la base de données.
2. **Langage commun universel** : Il contient l'ensemble des **DTOs** (*Data Transfer Objects*), des requêtes, des réponses et des énumérations échangées entre le Front et le Back.
3. **Sécurité à la compilation** : En partageant le même contrat immuable, si un champ est renommé dans `Shared`, le compilateur génère immédiatement une erreur de compilation dans le Front et dans le Back, empêchant tout bug de communication en production.

▶ Schéma Conceptuel & Flux : Shared : L'Autorité Centrale des Contrats d'Échange



Comparaison des Types de Projets dans la Solution

Projet	Type de Sortie	Dépendances Cibles	Rôle dans le Système
Shared	`.dll` (*Library*)	Aucune (pur .NET 8)	Contrats d'échange, DTOs, requêtes, enums.
Back	`.exe` (*Executable*)	Shared, EF Core, Kestrel	API REST, logique d'affaires, persistance.
Front	`.exe` / Package OS	Shared, Refit, MAUI	Interface graphique, interactions, panier local.

▶ Règle d'Ingénierie & Bonne Pratique : Immutabilité et Pureté des Contrats

Pour conserver un projet `Shared` sain et durable tout au long du cycle de vie logiciel :

- **Zéro dépendance lourde** : Ne jamais installer de package lié à la base de données (Entity Framework Core) ou à l'interface graphique dans `Shared`.
- **Modélisation par `record`** : Les DTOs doivent être des types immuables (`public record ...`) garantissant qu'une donnée ne peut pas être altérée accidentellement en cours de transfert.

▶ Questions Fréquentes du Débutant : Questions Fondamentales du Débutant

1. **Pourquoi ce projet ne produit-il pas de fichier `.exe` ? Comment s'exécute-t-il ?**
   Réponse : Une bibliothèque de classes (`classlib`) produit exclusivement un fichier `.dll`. Elle ne possède pas de point d'entrée et ne peut pas être « lancée » toute seule. Elle est conçue pour être chargée en mémoire par des projets exécutables (`Back` et `Front`) lors de leur propre démarrage.
2. **Pourquoi appelle-t-on ce projet `Shared` plutôt que `Common` ou `Core` ?**
   Réponse : Le nom `Shared` indique sans ambiguïté qu'il s'agit du contrat **partagé** entre deux mondes distincts (client et serveur). Le terme `Core` désigne habituellement le cœur métier (entités de domaine, logique interne du serveur), tandis que `Common` est un nom souvent déconseillé car il devient rapidement un « dossier fourre-tout » de code désorganisé.
3. **Pourquoi ne pas mettre Entity Framework Core et les tables de base de données dans `Shared` ?**
   Réponse : Ce serait une **faille architecturale et de sécurité majeure**. Si `Front` référençait les entités de base de données, l'application mobile embarquerait inutilement les pilotes SQL serveur et risquerait de tenter un accès direct aux données. Dans une application Fullstack professionnelle, le client mobile ne doit **jamais** connaître la base de données centrale : il ne dialogue avec le monde extérieur que via des requêtes HTTP transportant des DTOs.

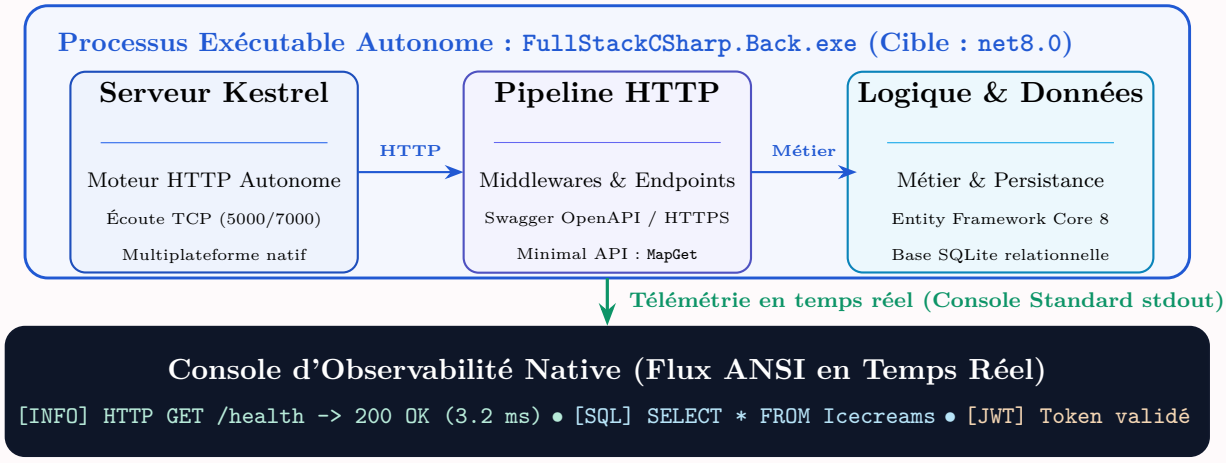
1.3 Action 1.3 : Création de l'API Backend (ASP.NET Core Minimal API)

► Concept Clé d'Architecture : Le Serveur Web Kestrel & le Paradigme des Minimal APIs

Le deuxième projet créé dans notre solution est `FullStackCSharp.Back`.
Contrairement aux bibliothèques de classes, ce projet est configuré avec le SDK Web (`Microsoft.NET.Sdk.Web`) et compile directement vers un exécutable autonome (`FullStackCSharp.Back.exe`).
Dans l'écosystème .NET moderne (.NET 8 LTS), la conception du Backend repose sur deux piliers majeurs :

1. **Serveur Kestrel intégré & Console Native** : Kestrel est un serveur web HTTP asynchrone haute performance embarqué directement dans le binaire. L'application se lance en ligne de commande via `dotnet run` et ouvre instantanément une console d'observabilité diffusant en direct les requêtes réseau, les temps de réponse et les requêtes SQL.
2. **Minimal APIs** : Introduit pour éliminer la lourdeur des contrôleurs MVC traditionnels, le modèle Minimal API permet de déclarer les routes HTTP directement via des lambdas typées (`app.MapGet`, `app.MapPost`) avec une allocation mémoire minimale et des performances brutes décuplées.

► Schéma Conceptuel & Flux : L'Anatomie du Processus Exécutable & la Console d'Observabilité



Comparatif d'Ingénierie : Minimal APIs vs Contrôleurs MVC

Critère	Contrôleurs MVC Classiques	Minimal APIs (.NET 8 LTS)
Style de déclaration	Classes dérivées de `ControllerBase` avec attributs	Fonctions lambdas directes (`app.MapGet`, etc.)
Consommation mémoire	Plus élevée (réflexion, filtres, conventions)	Minimale (zéro instanciation de contrôleur par requête)
Vitesse de démarrage	Chargement et scan d'assemblies au démarrage	Enregistrement immédiat dans le graphe de routage
Plafond de code	Fichiers souvent verbeux (> 300 lignes)	Fichiers modulaires ultra-compacts (< 100 lignes)
Documentation OpenAPI	Nécessite de nombreux attributs décoratifs	Intégration fluide avec `.WithOpenApi()` et TypedResults

▶ Règle d'Ingénierie & Bonne Pratique : Organisation Modulaire & Règle des 300 Lignes

Même avec les Minimal APIs, il ne faut jamais accumuler tous les endpoints dans un seul fichier `Program.cs` sous peine de créer un fichier géant ingérable.
Dans notre architecture, nous appliquons deux règles d'ingénierie strictes :

- **Fichiers d'extensions dédiés** : Chaque domaine métier dispose de son propre fichier statique d'extension (ex : `CatalogEndpoints.cs`, `AuthEndpoints.cs`, `OrderEndpoints.cs`).
- **Plafond strict de 300 lignes** : Chaque composant reste concis, lisible et focalisé sur une responsabilité unique.

▶ Questions Fréquentes du Débutant : Questions Fondamentales du Débutant

1. **Pourquoi une API Web produit-elle un fichier `.exe` ? Où se trouve le serveur web ?**

   Réponse : Contrairement aux anciennes technologies (comme ASP.NET sous IIS) où l'application était une bibliothèque hébergée dans un serveur tiers, ASP.NET Core moderne est **auto-hébergé** (*self-hosted*). Le serveur Kestrel fait partie intégrante de votre application. Le fichier `.exe` est donc le serveur web lui-même : un simple double-clic ou la commande `dotnet run` démarre le serveur et commence à écouter les requêtes réseau.

2. **Pourquoi dit-on que le Back est aussi la « console d'observabilité » ?**

   Réponse : En lançant `dotnet run` dans le dossier `FullStackCSharp.Back`, la console de votre terminal devient un tableau de bord vivant. Grâce à la coloration ANSI native de .NET, vous voyez défiler en temps réel :
   - Chaque requête HTTP entrante avec sa méthode, son URL et son code de retour (ex : `200 OK`, `401 Unauthorized`).
   - Les requêtes SQL réelles exécutées par Entity Framework Core sur la base SQLite locale.
   - Les étapes de validation des jetons de sécurité JWT et les contrôles anti-fraude.

   Il n'y a donc pas besoin d'un projet console supplémentaire : le serveur Kestrel offre une transparence totale pour l'apprentissage.

3. **Pourquoi utiliser les Minimal APIs plutôt que les Contrôleurs classiques ?**

   Réponse : Les Minimal APIs représentent le standard moderne recommandé par Microsoft pour les microservices et les APIs REST légères. Elles réduisent drastiquement le code superflu (*boilerplate*), accélèrent le temps de réponse et permettent à un développeur débutant de comprendre le flux HTTP d'un seul coup d'œil sans se perdre dans la hiérarchie complexe de contrôleurs abstraits.

4. **À quoi servent les adresses (comme `localhost:5000` ou `localhost:7000`) ? Comment le client mobile y accèdera-t-il ?**

   Réponse : Ces adresses définissent les ports TCP sur lesquels Kestrel écoute sur votre machine de développement. Sur un ordinateur, l'application mobile de bureau accède directement à `localhost`. Lorsque nous testerons sur un émulateur Android (dans un jalon ultérieur), l'émulateur utilise une passerelle réseau spéciale (`10.0.2.2`) pour joindre le serveur hôte.